

# Des mathématiques au fonctionnement d'une IA

## Table des matières

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Vue d'ensemble</b>                                            | <b>2</b> |
| <b>2</b> | <b>Autour des mots, des phrases, des textes...</b>               | <b>2</b> |
| 2.1      | Du texte aux vecteurs : espaces vectoriels et matrices . . . . . | 2        |
| 2.2      | Le mécanisme d'attention : produit scalaire . . . . .            | 4        |
| 2.3      | Position des mots : rotations . . . . .                          | 4        |
| <b>3</b> | <b>Les réseaux de neurones</b>                                   | <b>4</b> |
| 3.1      | De l'entrée à la sortie : un neurone comme fonction . . . . .    | 4        |
| 3.2      | Fonctions d'activation usuelles . . . . .                        | 5        |
| 3.3      | Empiler des neurones : le réseau de neurones . . . . .           | 6        |
| 3.4      | Retranscription : probabilités . . . . .                         | 7        |
| <b>4</b> | <b>Comment un réseau « apprend »-il ?</b>                        | <b>7</b> |
| 4.1      | Fonction de coût . . . . .                                       | 7        |
| 4.2      | Descente de gradient : dérivation et optimisation . . . . .      | 7        |
| 4.3      | Rétro-propagation : dérivation . . . . .                         | 8        |
| <b>5</b> | <b>En résumé</b>                                                 | <b>8</b> |

Bienvenue en MP2I !

Vous allez, cette année, apprendre à manipuler des nombres complexes, des espaces vectoriels, des matrices, des dérivées, des primitives, des probabilités, des suites, des algorithmes... des objets qui peuvent sembler abstraits et parfois distants de la Physique ou de l'Informatique.

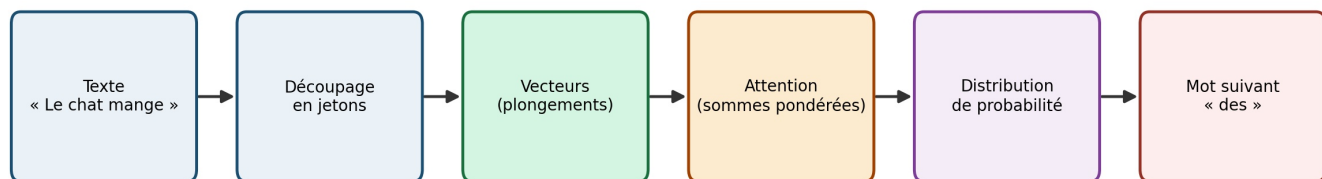
Nous allons tout au cours de l'année essayer de faire le lien entre les concepts mathématiques que nous aborderons et l'informatique. Notre fil conducteur sera l'intelligence artificielle générative (ChatGPT, Claude, et les autres grands modèles de langage, ou **LLM** pour *Large Language Model*).

Vous allez voir que les mathématiques sont fondamentales pour comprendre ces systèmes !

## 1 Vue d'ensemble

Un modèle de langage comme Claude ou ChatGPT peut être vu comme une **fonction mathématique gigantesque** qui prend en entrée un texte (votre question) et produit en sortie, jeton par jeton (que l'on appelle des **Tokens**), le texte le plus plausible qui pourrait suivre.

Ainsi le problème est : comment prédire le mot suivant connaissant tout le contexte précédent (mots, phrases, documents, etc...) ?



Chacune de ces étapes, et le réglage des paramètres du modèle, s'appuient sur des notions mathématiques précises que l'on retrouve dans le programme officiel des deux années de CPGE.

## 2 Autour des mots, des phrases, des textes...

### 2.1 Du texte aux vecteurs : espaces vectoriels et matrices

Les ordinateurs ne manipulent pas directement les mots. Chaque mot est transformé en un **vecteur** de grande dimension. C'est ce que l'on appelle le **plongement** (*embedding*).

Exemple.

$$professeur \longrightarrow \begin{pmatrix} 0.15 \\ -0.32 \\ \vdots \\ 0.81 \end{pmatrix}$$

Tous les mots appartiennent au même **espace vectoriel**  $\mathbb{R}^d$  ou  $\mathcal{M}_{d,1}(\mathbb{R})$

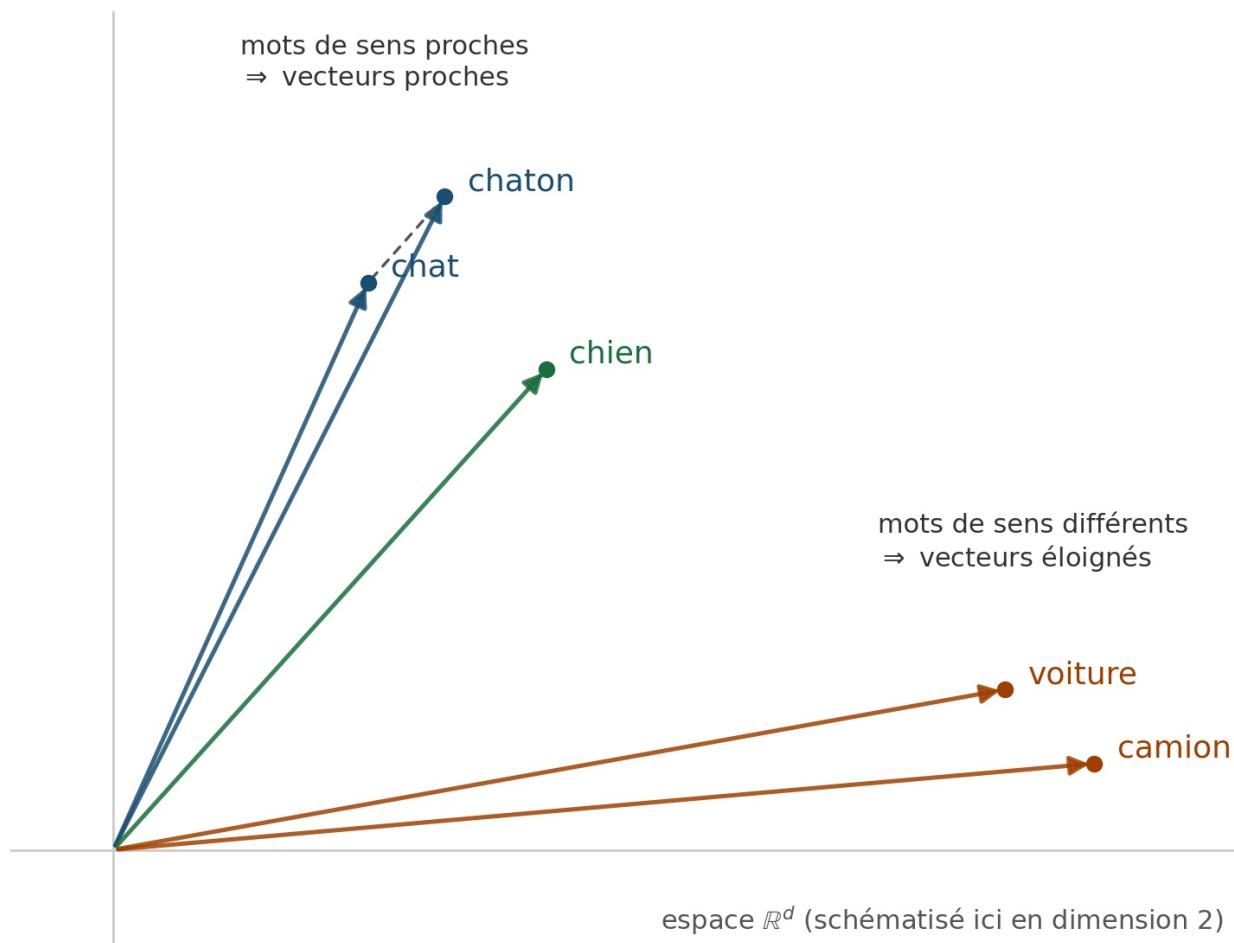
Si l'on considère que chaque mot est codé par un seul token, une phrase contenant  $n$  mots est représentée par une **matrice**  $X$  appartenant à  $\mathcal{M}_{n,d}(\mathbb{R})$  (l'ensemble des matrices à  $n$  lignes et  $d$  colonnes). Ainsi chaque ligne correspond à un mot.

Tout le texte devient une grande matrice !

Remarques.

- Dans ChatGPT2, on avait  $d = 768$  et dans ChatGPT 3 on avait  $d = 12288$ . La valeur de  $d$  pour la dernière version de ChatGPT n'est pas connue.
- Les IA de type LLM utilisent des **tokens** pour découper certains mots. Par exemple le mot "professeur" est découpé en un seul token, puis codé en un seul vecteur de  $\mathbb{R}^d$ . Mais le mot "extraordinaire" est probablement codé en deux tokens correspondant à "extra" et "ordinaire", puis codé avec deux vecteurs de  $\mathbb{R}^d$  (mais la tokenisation est tenue secrète par les géants de l'IA...)

Le choix des coordonnées des mots est fait afin que deux mots de sens proche aient des coordonnées relativement similaires.



Remarque. Le schéma précédent permet de comprendre la « proximité » des mots, mais il peut fausser notre perception. En effet, ici la représentation se fait en dimension 2 alors que dans une IA générative plusieurs milliers de coordonnées permettent de coder un mot. Cela rend les « proximités » de mots beaucoup plus riches.

## 2.2 Le mécanisme d'attention : produit scalaire

Le **mécanisme d'attention** est l'une des idées les plus importantes de l'IA moderne. Son principe est de permettre à un modèle de **se concentrer dynamiquement** sur les parties les plus pertinentes d'une entrée, plutôt que de traiter toutes les informations de la même manière.

Les couches d'attention se situent dans les réseaux de neurones.

*Exemple.* Dans la phrase "Le chat, qu'elle a adopté hier, dort", pour bien traiter "dort", un modèle doit se souvenir que le sujet est "le chat", malgré la longue proposition intercalée. L'attention permet d'aller chercher directement l'information "chat", où qu'elle soit dans la phrase, sans passer par une chaîne d'états intermédiaires.

Mathématiquement, le calcul porte sur des **produits scalaires** entre vecteurs. L'objectif est ensuite d'obtenir des **probabilités**.

## 2.3 Position des mots : rotations

Un dernier problème : le mécanisme d'attention, tel que décrit ci-dessus, traite tous les tokens de façon symétrique et ne « sait » donc pas dans quel ordre ils apparaissent dans la phrase ! Il faut lui injecter une information de **position**. Sans entrer dans les détails, on effectue des **rotations** sur les vecteurs de dimension  $d$ . Les objets mathématiques sous-jacents sont des **matrices**.

# 3 Les réseaux de neurones

## 3.1 De l'entrée à la sortie : un neurone comme fonction

L'idée fondatrice de l'intelligence artificielle est de s'inspirer, très librement, du neurone biologique : une cellule qui reçoit plusieurs signaux d'entrée, les combine, et déclenche (ou non) un signal de sortie.

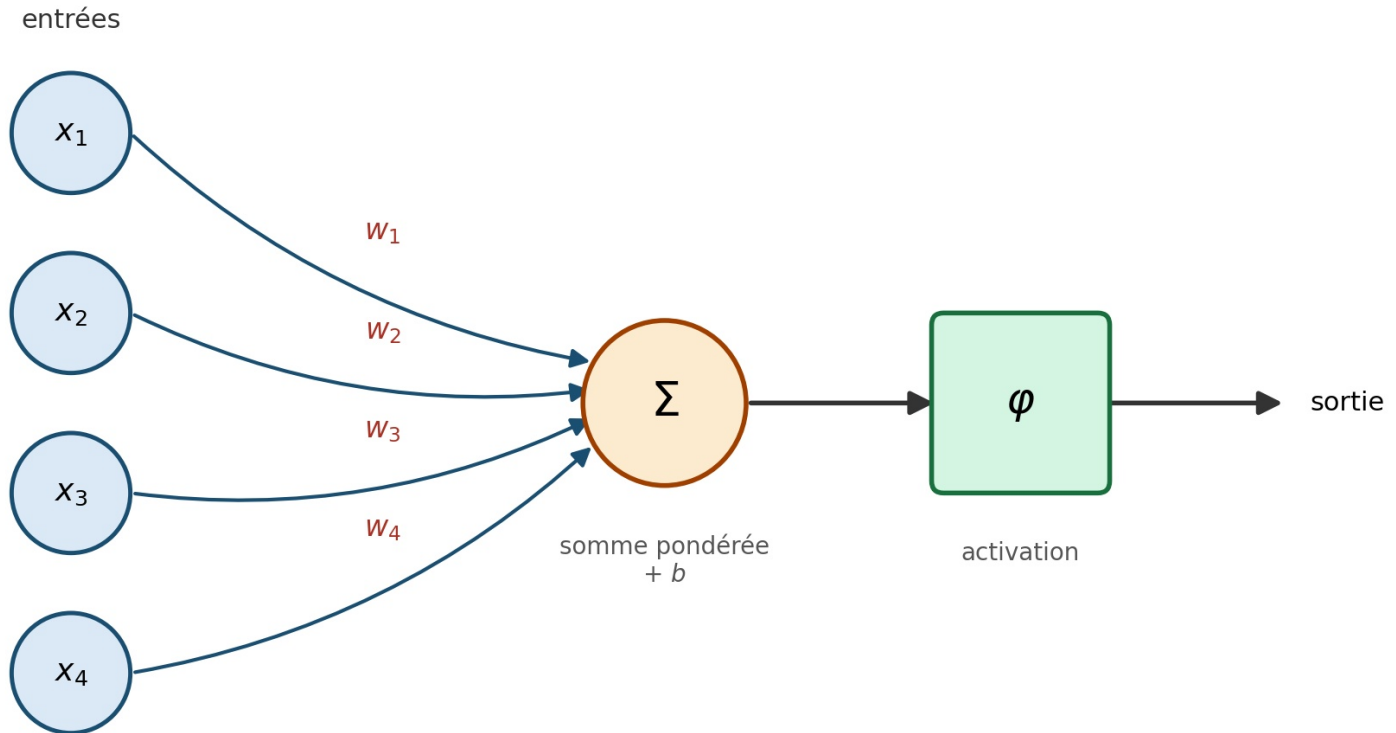
**DÉFINITION. (NEURONE FORMEL)**

Un **neurone formel** à  $n$  entrées ( $n \in \mathbb{N}^*$ ) est une fonction  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  définie, pour  $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ , par

$$f(x) = \varphi \left( \sum_{i=1}^n w_i x_i + b \right) = \varphi(\langle w, x \rangle + b),$$

où  $w = (w_1, \dots, w_n) \in \mathbb{R}^n$  est le vecteur des **poids**,  $b \in \mathbb{R}$  est le **biais**, et  $\varphi : \mathbb{R} \rightarrow \mathbb{R}$  est une fonction non linéaire appelée **fonction d'activation**.

*Remarque.* Le terme  $\langle w, x \rangle = \sum_{i=1}^n w_i x_i$  est exactement le **produit scalaire canonique** de  $\mathbb{R}^n$  que vous rencontrerez au chapitre sur les espaces préhilbertiens. « **Apprendre** », pour ce neurone, consistera précisément à ajuster les coefficients  $w_i$  et  $b$  afin que le résultat de sortie se rapproche le plus du résultat attendu (dans la phase d'entraînement).



### 3.2 Fonctions d'activation usuelles

Sans la fonction  $\varphi$ , un neurone ne serait qu'une fonction affine, et une composition de fonctions affines resterait affine : empiler des neurones purement linéaires ne donnerait rien de plus puissant qu'un seul neurone linéaire ! C'est la **non-linéarité** de  $\varphi$  qui donne toute sa puissance au réseau.

Voici quelques exemples de fonctions d'activation :

| Nom                   | Expression                                         |
|-----------------------|----------------------------------------------------|
| Sigmoïde              | $\sigma : x \mapsto \frac{1}{1 + e^{-x}}$          |
| Tangente hyperbolique | $th : x \mapsto \frac{e^x - e^{-x}}{e^x + e^{-x}}$ |
| ReLU                  | $ReLU : x \mapsto \max(x, 0)$                      |
| GELU                  | $GELU : x \mapsto x\Phi(x)$                        |

La fonction d'activation Softmax est un peu particulière et est utilisée en sortie du réseau de neurones.

Pour  $x = (x_1, \dots, x_n) \in \mathbb{R}^n$  en entrée elle renvoie  $y = (y_1, \dots, y_n) \in \mathbb{R}^n$  telle que pour tout entier relatif compris entre 1 et  $n$ ,

$$y_i = \text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{k=1}^n e^{x_k}}$$

On obtient ainsi une **distribution de probabilités**.

### 3.3 Empiler des neurones : le réseau de neurones

#### DÉFINITION. (COUCHE DENSE, RÉSEAU DE NEURONES)

Une **couche dense** (ou couche « entièrement connectée ») de  $p$  neurones à  $n$  entrées est une fonction  $\mathbb{R}^n \rightarrow \mathbb{R}^p$  qui s'écrit, en notant  $W \in \mathcal{M}_{p,n}(\mathbb{R})$  la **matrice des poids** et  $b \in \mathbb{R}^p$  le vecteur des biais,

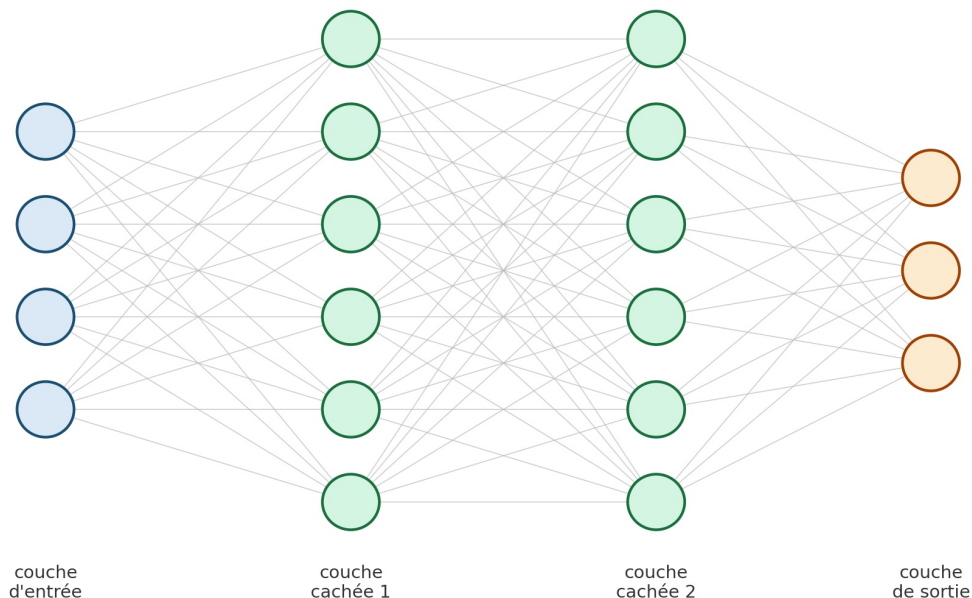
$$x \mapsto \varphi(Wx + b),$$

où  $\varphi$  s'applique coordonnée par coordonnée. Un **réseau de neurones** à  $K$  couches (perceptron multicouche) s'obtient en composant plusieurs couches :

$$x \mapsto f_K \circ f_{K-1} \circ \cdots \circ f_1(x).$$

#### Informatique

C'est pour cette raison que les réseaux de neurones se calculent extrêmement bien sur des **cartes graphiques (GPU)** : un GPU est une machine conçue pour effectuer des **produits matriciels** en parallèle.



*Remarque.* Les neurones sont très riches d'un point de vue mathématique. Ils font intervenir de nombreuses notions du programme :

- produit matriciel et produit scalaire,
- calculs de sommes,
- étude de fonctions (fonctions d'activation)

### 3.4 Retranscription : probabilités

En sortie de réseau, on obtient plusieurs vecteurs de sortie, c'est à dire des vecteur de  $\mathbb{R}^n$ , qui correspondent à des mots (ou plus précisément des *token*). La **retranscription** (*unembedding*) est l'étape finale : les vecteurs sont « traduits » en **probabilités** pour choisir le prochain mot.

Contrairement à *l'embedding* d'entrée qui transforme des mots en vecteurs, *l'unembedding* fait l'inverse. Il prend l'état de la dernière couche du réseau de neurones et le projette sur l'ensemble du vocabulaire du modèle pour prédire la suite du texte.

## 4 Comment un réseau « apprend »-il ?

### 4.1 Fonction de coût

Un réseau de neurones, une fois ses poids fixés, est une fonction parfaitement déterministe.

« Apprendre » signifie choisir les poids  $w$  (on regroupe tous les poids et biais de toutes les couches sous ce même nom) de sorte que le réseau reproduise, le mieux possible, un comportement souhaité observé sur des exemples.

Pour cela, pendant son entraînement, on lui montre des milliards de phrases. Il évalue alors l'erreur, et il cherche à la minimiser en modifiant les paramètres : c'est un problème **d'optimisation**.

#### DÉFINITION. (FONCTION DE COÛT)

Soit  $(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})$ , avec  $N \in \mathbb{N}^*$ , un jeu de données d'exemples (entrée, sortie souhaitée), et soit  $f_w$  le réseau de neurones de poids  $w$ . On appelle **fonction de coût** (ou de perte) une fonction  $\mathcal{L}$  mesurant l'écart entre les prédictions  $f_w(x^{(k)})$  et les sorties souhaitées  $y^{(k)}$ .

On peut par exemple utiliser l'**erreur quadratique moyenne** :

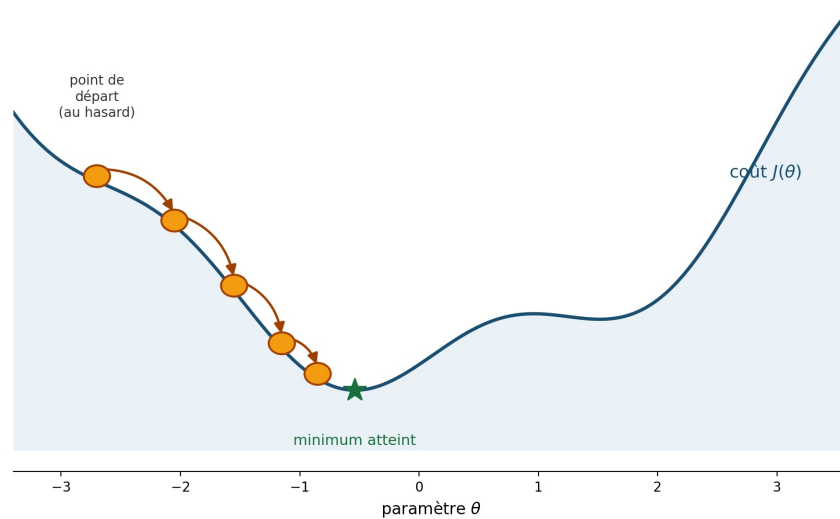
$$\mathcal{L}(w) = \frac{1}{N} \sum_{k=1}^N \|f_w(x^{(k)}) - y^{(k)}\|^2.$$

Apprendre, c'est chercher  $w$  qui **minimise**  $\mathcal{L}(w)$ .

### 4.2 Descente de gradient : dérivation et optimisation

Minimiser une fonction  $\mathcal{L}$  de quelques milliards de variables réelles est hors de portée du calcul formel.

On procède donc de façon **itérative** avec l'algorithme de descente de gradient, que vous découvrirez en MPI.



### 4.3 Rétro-propagation : dérivation

Reste une difficulté de taille : un réseau profond est une composition de dizaines, voire de centaines de couches. Comment calculer, efficacement, le gradient de  $\mathcal{L}$  par rapport à **chacun** des milliards de poids du réseau ?

Pour un réseau  $f_K \circ \dots \circ f_1$ , on calcule le gradient de proche en proche, de la **dernière couche vers la première** (d'où le nom de rétro-propagation), chaque couche transmettant à la précédente le produit de son propre gradient par celui reçu de la couche suivante.

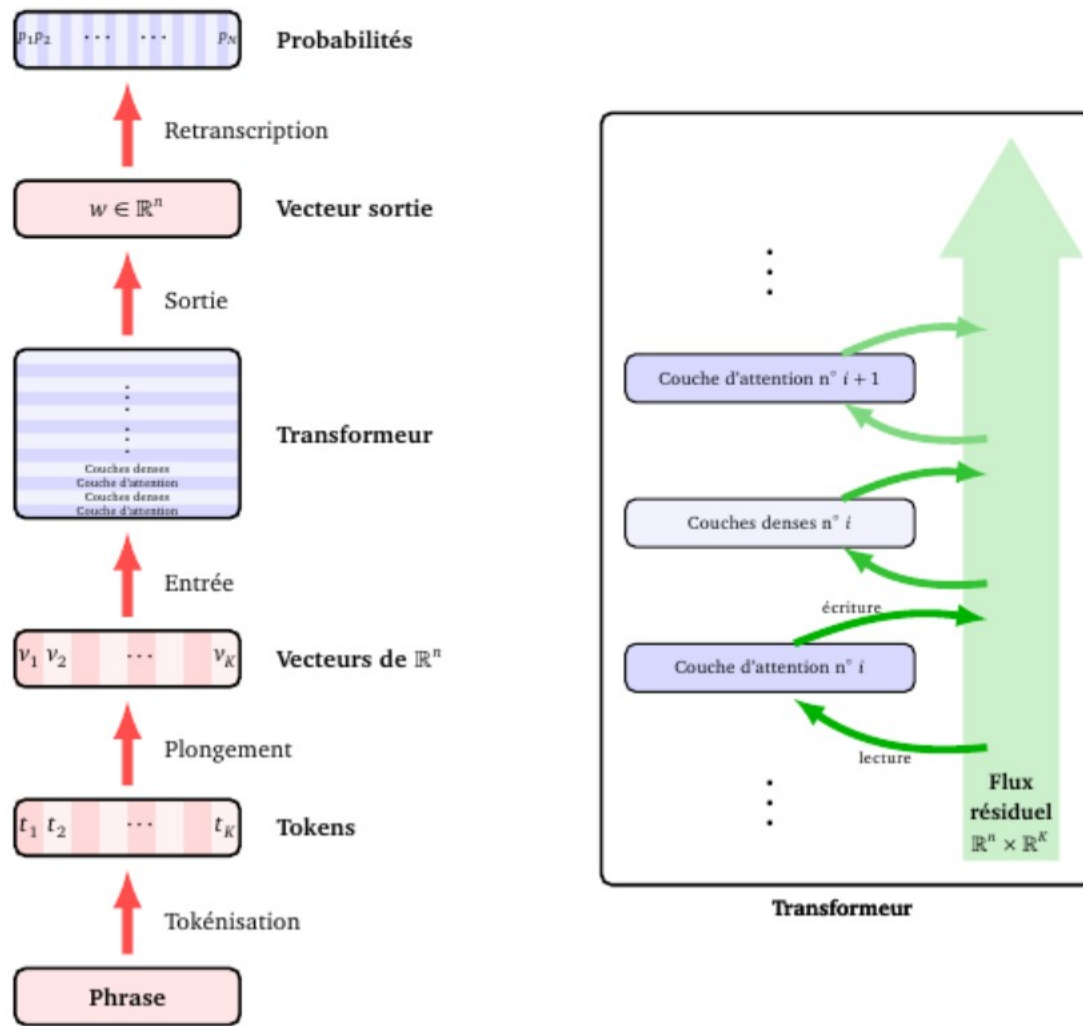
Exemple. Si  $f$  et  $g$  sont deux fonctions réelles dérivables et que l'on cherche à dériver  $g \circ f$ , on sait que

$$(g \circ f)' = (g' \circ f) \times f'$$

C'est à petite échelle le phénomène de rétro-propagation du gradient.

## 5 En résumé

Nous avons vu l'architecture générale d'un IA générative, que nous pourrions résumer avec le schéma suivant :



Ce que l'on peut retenir :

- Une IA n'a **rien de magique** : c'est un très grand calcul, fait de briques simples répétées des milliards de fois. Il n'y a pas de « compréhension » au sens humain.
- Il faut avoir conscience que la **masse de données** qui permet aux IA d'« apprendre » dépasse les capacités de perceptions humaines. C'est cette masse de données qui donne au système sa pertinence. Ainsi une IA manipule des régularités statistiques à très grande échelle, avec toutes les limites que cela implique (biais des données, absence de vérité garantie, etc.).
- « **Apprendre** », pour une IA, signifie ajuster des réglages internes pour se tromper de moins en moins souvent.

*Remarque. Si vous vous intéressez au fonctionnement des IA, je vous invite à télécharger gratuitement cet excellent ouvrage (dont je me suis fortement inspiré...) à l'adresse suivante*

*<https://exo7math.github.io/deepmath-exo7/livre-deepmath.pdf>*